DEVI AHILYA VISHWAVIDYALAYA (DAVV), INDORE

School of Computer Science & Information Technology

Department of Computer Science & Information Technology

Programme: M.Tech (Information Architecture & Software Engineering)

LABORATORY MANUAL

Advanced Database Management Systems

Course Code: CS-6418

Semester 1, Year 1

Submitted By:

Name: ROHIT BHURE

Roll No.: 2610611

Programme: M.Tech (IA&SE)

Semester: 1 (Year 1)

Submitted To:

Name: Dr. Tarjani Sevak

Designation: Professor

Department: School of Computer Science & Information Technology

_____

Signature

Academic Session: 2026–2027

CERTIFICATE

This is to certify that ROHIT BHURE, Roll No. 2610611, has satisfactorily completed all the experiments of Advanced Database Management Systems (Course Code: CS-6418) as prescribed by Devi Ahilya Vishwavidyalaya (DAVV) for M.Tech (Information Architecture & Software Engineering), Semester 1, during the academic session 2026–2027.

_____ _____

Faculty / Lab Incharge
Name: Dr. Tarjani Sevak
Date: _____

INDEX

Exp. No.	Title of Experiment	Page No.	Date	Signature
1	Database Requirement Analysis	3		
2	ER Model Design	5		
3	ER to Relational Mapping	7		
4	Database Creation	9		
5	SQL Query Implementation	15		
6	Relational Algebra	17		
7	Advanced SQL Programming	19		
8	Database Integrity and Triggers	22		
9	Transaction Processing	25		
10	MongoDB CRUD Operations	27		
11	MongoDB Aggregation Framework	29		
12	Graph Database using Neo4j	31		
13	Spatial Database using PostGIS	33		

Experiment No. 1

Database Requirement Analysis

Aim:

To select a real-world application (Digital Library), gather its requirements, identify entities, attributes, relationships and business rules, and prepare a requirement specification document.

Software / Hardware Required:

Digital Library case study; MS Word / text editor for the SRS, draw.io or Graphviz for diagrams; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Requirement analysis is the first phase of database design. The designer studies the working of the organisation, interviews the users and collects the data that must be stored, the operations to be performed and the rules that the data must obey. The result is a Software Requirement Specification (SRS) that lists the entities (things about which data is stored), their attributes, the relationships between them and the business rules (constraints of the organisation). A clear SRS avoids redesign in the later phases: conceptual design (ER model), logical design (relational schema) and physical design (SQL).

Procedure / Algorithm:

1. Select the application: Digital Library of a university.
2. Gather the requirements by studying the manual library procedure and listing the users (members, librarians) and their needs.
3. Identify the entities and their attributes.
4. Identify the relationships with their cardinalities.
5. List the business rules and write the requirement specification document.

Requirement Specification Document:

REQUIREMENT SPECIFICATION - DIGITAL LIBRARY MANAGEMENT SYSTEM

1. Purpose

To maintain the books, members and the issue / return transactions of a university digital library and to reduce manual record keeping.

2. Users

Librarian (issues and receives books, collects fine), Student Member, Faculty Member and Administrator.

3. Functional Requirements

ID	Requirement
FR1	Store the details of books, authors, publishers and categories.
FR2	Register students and faculty as library members.
FR3	Issue a book to a member and record the due date.
FR4	Accept the returned book and update the available copies.
FR5	Calculate and store the fine for late returns.
FR6	Generate reports: pending returns, most issued books, member history.

4. Entities and Attributes

Entity	Attributes	Key
BOOK	book_id, isbn, title, price, copies, available_copies, pub_year, status	book_id
AUTHOR	author_id, name, country	author_id
PUBLISHER	publisher_id, name, city, email	publisher_id
CATEGORY	category_id, cat_name	category_id
MEMBER	member_id, name, email, phone, join_date, member_type, status	member_id
STUDENT (sub-class)	roll_no, course	member_id

Entity	Attributes	Key
FACULTY (sub-class)	department	member_id
LIBRARIAN	librarian_id, name, shift, salary	librarian_id
ISSUE_RECORD	issue_id, issue_date, due_date, return_date	issue_id
FINE (weak entity)	fine_no, amount, paid_status	issue_id + fine_no

5. Relationships

Relationship	Entities	Cardinality	Participation
Publishes	Publisher - Book	1 : N	Book total
Belongs to	Category - Book	1 : N	Book total
Written by	Book - Author	M : N	Book total
Issue	Member - Book	M : N	Partial
Handles	Librarian - Issue	1 : N	Issue total
Has	Issue_Record - Fine	1 : N	Fine total

6. Business Rules

ID	Rule
BR1	A member may have at most 3 books issued at a time.
BR2	A book is issued for 14 days; the due date is not earlier than the issue date.
BR3	Fine is Rs. 2 per day after the due date.
BR4	The ISBN of every book and the e-mail of every member must be unique.
BR5	A book can be issued only if available_copies > 0.
BR6	Price must be positive and copies cannot be negative.
BR7	Member type is either Student or Faculty.

7. Non-functional Requirements

Data integrity, multi-user access, daily backup and response time below 2 seconds.

Output:

The requirement specification document for the Digital Library (given above) was prepared. It identifies 9 entities, 7 relationships and 7 business rules, which are used as input for the ER design in the next experiment.

Result:

The requirement specification document for the Digital Library system was prepared, listing the entities, attributes, relationships and business rules.

Conclusion:

Requirement analysis gives a clear and complete basis for database design and avoids changes in the later phases.

Experiment No. 2

ER Model Design

Aim:

To create an ER model for the Digital Library including strong and weak entities, generalization, specialization, aggregation, cardinality and participation constraints.

Software / Hardware Required:

Digital Library case study; MS Word / text editor for the SRS, draw.io or Graphviz for diagrams; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

The Entity-Relationship (ER) model describes the data as entities, attributes and relationships. A strong entity has its own primary key, whereas a weak entity depends on an owner entity and is identified by the owner's key plus a partial key (double rectangle). Specialization is the top-down process of defining sub-classes (STUDENT, FACULTY) of an entity (MEMBER); generalization is the reverse bottom-up process. Aggregation treats a relationship together with its entities as a higher-level entity so that it can take part in another relationship. Cardinality (1:1, 1:N, M:N) shows how many instances take part, and participation is total (double line) or partial (single line).

Procedure / Algorithm:

1. Take the entities and relationships identified in Experiment 1.
2. Draw strong entities (Book, Author, Publisher, Category, Member, Librarian) with their attributes and underline the key attributes.
3. Draw the weak entity Fine (double rectangle) with the identifying relationship Has (double diamond) and the partial key fine_no.
4. Show the specialization of Member into Student and Faculty using an ISA triangle (disjoint, total).
5. Show the aggregation of Member-Issue-Book (dashed box) and connect it with Librarian through the relationship Handles.
6. Mark the cardinality (1, M, N) on every relationship and double lines for total participation.

ER Model Description:

Constructs used in the ER model

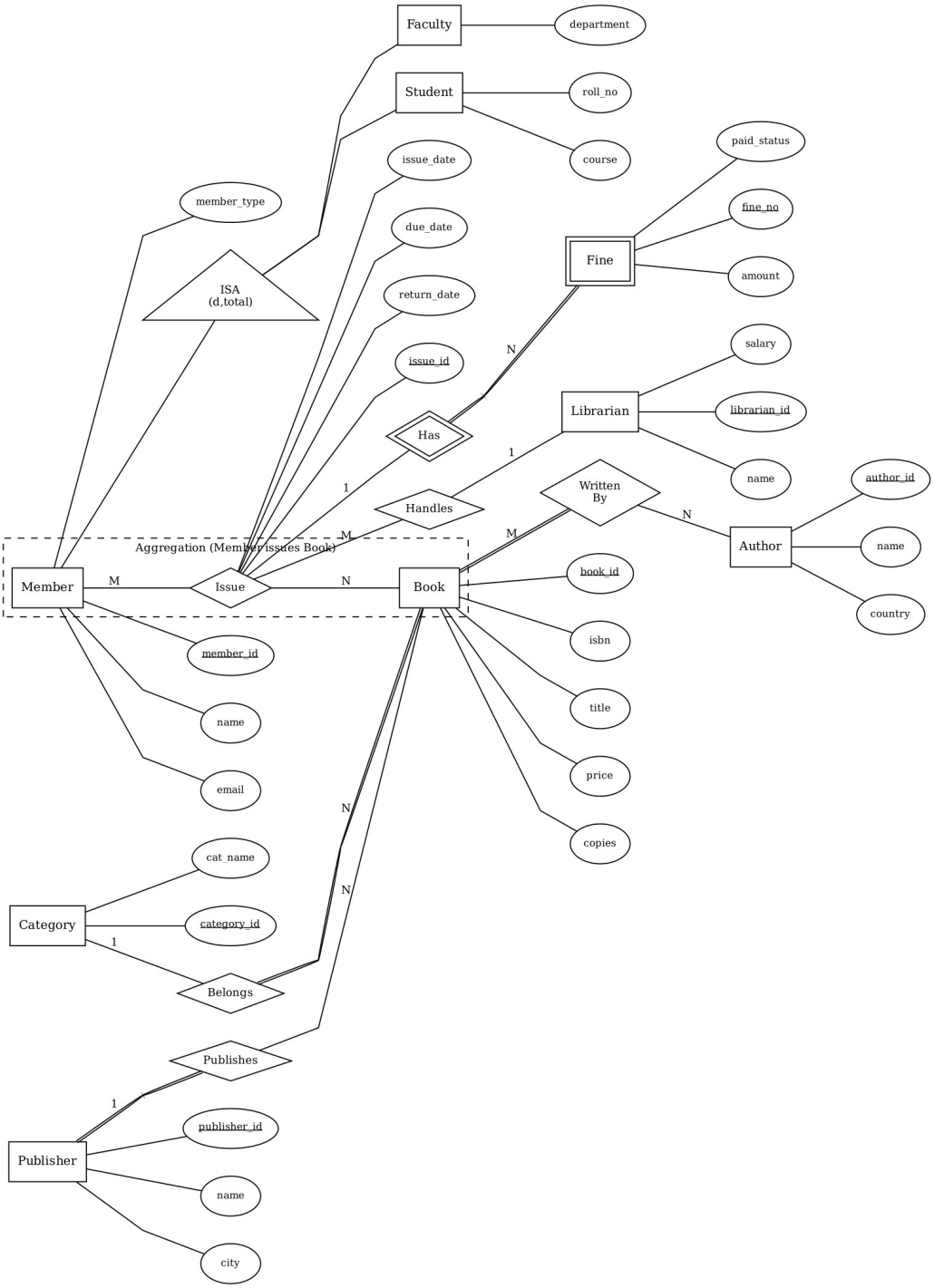
Construct	Description
Strong entities	Publisher, Category, Book, Author, Member, Librarian
Weak entity	Fine (owner: Issue_Record, partial key: fine_no)
Specialization	Member into Student and Faculty (disjoint, total)
Generalization	Student and Faculty generalized to Member (reverse of the above)
Aggregation	The relationship Member-Issue-Book is treated as one entity and related to Librarian through Handles

Relationships, cardinality and participation

Relationship	Entities	Cardinality	Participation
Publishes	Publisher - Book	1 : N	Book total (double line)
Belongs	Category - Book	1 : N	Book total (double line)
Written By	Book - Author	M : N	Book total (double line)
Issue	Member - Book	M : N	Partial (single line)
Handles	Issue (aggregate) - Librarian	N : 1	Issue total
Has	Issue - Fine	1 : N	Fine total (identifying)

Output:

ER diagram of the Digital Library (strong and weak entities, ISA specialization, aggregation, cardinalities and participation):



Result:

The ER model of the Digital Library was designed with all the required constructs.

Conclusion:

The ER model represents the real-world system clearly and can be converted directly into relational tables.

Experiment No. 3

ER to Relational Mapping

Aim:

To convert the ER model of the Digital Library into relational tables with primary keys, foreign keys, composite keys and candidate keys.

Software / Hardware Required:

Digital Library case study; MS Word / text editor for the SRS, draw.io or Graphviz for diagrams; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

An ER model is mapped to the relational model by fixed rules: (1) each strong entity becomes a table whose primary key is its key attribute; (2) a weak entity becomes a table whose primary key is the owner's key plus its partial key (composite key); (3) a 1:N relationship is mapped by adding the primary key of the '1' side as a foreign key in the 'N' side table; (4) an M:N relationship becomes a separate table whose primary key is the combination of both foreign keys; (5) for specialization, each subclass table uses the superclass key as primary key and foreign key. A candidate key is any minimal set of attributes that can uniquely identify a row; one of them is chosen as the primary key.

Procedure / Algorithm:

1. Map each strong entity to a table (Publisher, Category, Book, Author, Member, Librarian).
2. Map the specialization: Student and Faculty tables with member_id as PK and FK.
3. Map the weak entity Fine with the composite key (issue_id, fine_no).
4. Map 1:N relationships by adding foreign keys (Book.category_id, Book.publisher_id).
5. Map the M:N relationship Written By to Book_Author with composite key (book_id, author_id).
6. Map the aggregation Issue to Issue_Record with foreign keys to Member, Book and Librarian; list the candidate keys.

Relational Schema:

Table	Columns	Primary Key	Foreign Key
PUBLISHER	publisher_id, name, city, email	publisher_id	-
CATEGORY	category_id, cat_name	category_id	-
AUTHOR	author_id, name, country	author_id	-
BOOK	book_id, isbn, title, category_id, publisher_id, price, copies, available_copies, pub_year, status	book_id	category_id, publisher_id
BOOK_AUTHOR	book_id, author_id	(book_id, author_id)	book_id, author_id
MEMBER	member_id, name, email, phone, join_date, member_type, status	member_id	-
STUDENT	member_id, roll_no, course	member_id	member_id
FACULTY	member_id, department	member_id	member_id
LIBRARIAN	librarian_id, name, shift, salary	librarian_id	-
ISSUE_RECORD	issue_id, member_id, book_id, librarian_id, issue_date, due_date, return_date	issue_id	member_id, book_id, librarian_id
FINE	issue_id, fine_no, amount, paid_status	(issue_id, fine_no)	issue_id

Keys

Key type	Details
Primary keys	publisher_id, category_id, author_id, book_id, member_id, librarian_id, issue_id
Foreign keys	Book.category_id, Book.publisher_id, Student / Faculty.member_id, Issue_Record.member_id, book_id, librarian_id, Fine.issue_id, Book_Author.book_id and author_id
Composite keys	Book_Author (book_id, author_id); Fine (issue_id, fine_no)

Key type	Details
Candidate keys	Book: {book_id}, {isbn}; Member: {member_id}, {email}; Student: {member_id}, {roll_no}; Publisher: {publisher_id}, {name}; Category: {category_id}, {cat_name}

Output:

The 11 relational tables listed above were obtained from the ER model. All foreign keys reference the primary keys of their parent tables, and these tables are created in the next experiment.

Result:

The ER model was converted into 11 relational tables with primary, foreign, composite and candidate keys.

Conclusion:

Applying the mapping rules gives a normalised relational schema that preserves all relationships of the ER model.

Experiment No. 4

Database Creation

Aim:

To create the complete Digital Library database with PRIMARY KEY, FOREIGN KEY, CHECK, UNIQUE and DEFAULT constraints and to populate each major table with at least 25 records.

Software / Hardware Required:

MySQL Server 8.0, MySQL Workbench / MySQL command-line client; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

DDL (Data Definition Language) statements such as CREATE TABLE define the structure of the database. Constraints enforce integrity: PRIMARY KEY (unique and not null), FOREIGN KEY (value must exist in the parent table), CHECK (condition on the column value), UNIQUE (no duplicate values) and DEFAULT (value used when none is given). The INSERT statement of DML adds rows to a table. Tables must be created and populated in the order parent tables first, child tables afterwards so that foreign keys are satisfied.

Procedure / Algorithm:

1. Start MySQL and create the database digital_library.
2. Create the parent tables Publisher, Category, Author, Book and Book_Author with constraints.
3. Create Member, Student, Faculty, Librarian, Issue_Record and Fine tables.
4. Insert 25 records in each major table (Publisher, Author, Book, Member, Issue_Record) and the required rows in the other tables.
5. Verify the number of rows in each table with SELECT COUNT(*).

Code / Program:

```
CREATE DATABASE digital_library;
USE digital_library;

CREATE TABLE publisher (
  publisher_id INT PRIMARY KEY,
  name         VARCHAR(60) NOT NULL UNIQUE,
  city         VARCHAR(30),
  email        VARCHAR(60)
);
CREATE TABLE category (
  category_id INT PRIMARY KEY,
  cat_name    VARCHAR(30) NOT NULL UNIQUE
);
CREATE TABLE author (
  author_id INT PRIMARY KEY,
  name      VARCHAR(50) NOT NULL,
  country   VARCHAR(30)
);
CREATE TABLE book (
  book_id          INT PRIMARY KEY,
  isbn             VARCHAR(15) NOT NULL UNIQUE,          -- candidate key
  title            VARCHAR(80) NOT NULL,
  category_id      INT NOT NULL,
  publisher_id     INT NOT NULL,
  price            DECIMAL(8,2) NOT NULL CHECK (price > 0),
  copies           INT NOT NULL CHECK (copies >= 0),
  available_copies INT NOT NULL CHECK (available_copies >= 0),
  pub_year         INT CHECK (pub_year BETWEEN 1900 AND 2100),
  status           VARCHAR(12) DEFAULT 'Available',
  FOREIGN KEY (category_id)  REFERENCES category(category_id),
  FOREIGN KEY (publisher_id) REFERENCES publisher(publisher_id)
);
CREATE TABLE book_author (
  book_id   INT,
  author_id INT,
  PRIMARY KEY (book_id, author_id),                      -- composite key
```

```
  FOREIGN KEY (book_id)   REFERENCES book(book_id),
  FOREIGN KEY (author_id) REFERENCES author(author_id)
);
CREATE TABLE member (
  member_id   INT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL,
  email       VARCHAR(60) NOT NULL UNIQUE,              -- candidate key
  phone       VARCHAR(10),
  join_date   DATE NOT NULL,
  member_type VARCHAR(10) NOT NULL CHECK (member_type IN ('Student','Faculty')),
  status      VARCHAR(10) DEFAULT 'Active'
);
CREATE TABLE student (
  member_id INT PRIMARY KEY,
  roll_no   VARCHAR(10) NOT NULL UNIQUE,
  course    VARCHAR(20),
  FOREIGN KEY (member_id) REFERENCES member(member_id)
);
CREATE TABLE faculty (
  member_id  INT PRIMARY KEY,
  department VARCHAR(30),
  FOREIGN KEY (member_id) REFERENCES member(member_id)
);
CREATE TABLE librarian (
  librarian_id INT PRIMARY KEY,
  name         VARCHAR(50) NOT NULL,
  shift        VARCHAR(10) DEFAULT 'Morning',
  salary       DECIMAL(9,2) CHECK (salary > 0)
);
CREATE TABLE issue_record (
  issue_id     INT PRIMARY KEY,
  member_id    INT NOT NULL,
  book_id      INT NOT NULL,
  librarian_id INT NOT NULL,
  issue_date   DATE NOT NULL,
  due_date     DATE NOT NULL,
  return_date  DATE,
  CHECK (due_date >= issue_date),
  FOREIGN KEY (member_id)    REFERENCES member(member_id),
  FOREIGN KEY (book_id)      REFERENCES book(book_id),
  FOREIGN KEY (librarian_id) REFERENCES librarian(librarian_id)
);
CREATE TABLE fine (                                     -- weak entity
  issue_id    INT,
  fine_no     INT,
  amount      DECIMAL(7,2) NOT NULL CHECK (amount >= 0),
  paid_status VARCHAR(8) DEFAULT 'Unpaid',
  PRIMARY KEY (issue_id, fine_no),
  FOREIGN KEY (issue_id) REFERENCES issue_record(issue_id) ON DELETE CASCADE
);

-- Populate the tables (parents first)
INSERT INTO publisher (publisher_id, name, city, email) VALUES
(1, 'Pearson Education', 'Noida', 'info1@pearsonedu.com'),
(2, 'McGraw-Hill', 'New Delhi', 'info2@mcgrawhill.com'),
(3, 'Wiley India', 'New Delhi', 'info3@wileyindia.com'),
(4, 'Oxford University Press', 'New Delhi', 'info4@oxforduniv.com'),
(5, 'Cengage Learning', 'Mumbai', 'info5@cengagelea.com'),
(6, 'PHI Learning', 'New Delhi', 'info6@philearnin.com'),
(7, 'Tata McGraw-Hill', 'New Delhi', 'info7@tatamcgraw.com'),
(8, 'Springer', 'Berlin', 'info8@springer.com'),
(9, 'O''Reilly Media', 'Sebastopol', 'info9@oreillymed.com'),
(10, 'Addison-Wesley', 'Boston', 'info10@addisonwes.com'),
(11, 'MIT Press', 'Cambridge', 'info11@mitpress.com'),
(12, 'Cambridge University Press', 'Cambridge', 'info12@cambridgeu.com'),
(13, 'BPB Publications', 'New Delhi', 'info13@bpbpublica.com'),
(14, 'Khanna Publishing', 'New Delhi', 'info14@khannapubl.com'),
(15, 'Narosa Publishing', 'New Delhi', 'info15@narosapubl.com'),
(16, 'Elsevier India', 'Gurugram', 'info16@elsevierin.com'),
(17, 'Sapna Book House', 'Bengaluru', 'info17@sapnabookh.com'),
(18, 'Vikas Publishing', 'Noida', 'info18@vikaspubli.com'),
```

```
(19, 'S. Chand & Co', 'New Delhi', 'info19@schandco.com'),
(20, 'Laxmi Publications', 'New Delhi', 'info20@laxmipubli.com'),
(21, 'Dhanpat Rai', 'New Delhi', 'info21@dhanpatrai.com'),
(22, 'Packt Publishing', 'Birmingham', 'info22@packtpubli.com'),
(23, 'Manning Publications', 'Shelter Island', 'info23@manningpub.com'),
(24, 'Apress', 'New York', 'info24@apress.com'),
(25, 'Prentice Hall', 'Hoboken', 'info25@prenticeha.com');
INSERT INTO category (category_id, cat_name) VALUES
(1, 'Database'),
(2, 'Programming'),
(3, 'Networking'),
(4, 'Algorithms'),
(5, 'Operating Systems'),
(6, 'Software Engineering');
INSERT INTO author (author_id, name, country) VALUES
(1, 'A. Silberschatz', 'USA'),
(2, 'Henry F. Korth', 'USA'),
(3, 'S. Sudarshan', 'India'),
(4, 'Ramez Elmasri', 'USA'),
(5, 'Shamkant Navathe', 'USA'),
(6, 'Raghu Ramakrishnan', 'USA'),
(7, 'Johannes Gehrke', 'USA'),
(8, 'Jeffrey Ullman', 'USA'),
(9, 'C. J. Date', 'UK'),
(10, 'Hector Garcia-Molina', 'USA'),
(11, 'Jennifer Widom', 'USA'),
(12, 'Brian Kernighan', 'Canada'),
(13, 'Dennis Ritchie', 'USA'),
(14, 'Andrew Tanenbaum', 'Netherlands'),
(15, 'Thomas Cormen', 'USA'),
(16, 'Donald Knuth', 'USA'),
(17, 'Robert Sedgewick', 'USA'),
(18, 'William Stallings', 'USA'),
(19, 'Behrouz Forouzan', 'Iran'),
(20, 'E. Balagurusamy', 'India'),
(21, 'Herbert Schildt', 'USA'),
(22, 'Joshua Bloch', 'USA'),
(23, 'Kathy Sierra', 'USA'),
(24, 'Erich Gamma', 'Switzerland'),
(25, 'Robert Martin', 'USA');
INSERT INTO book (book_id, isbn, title, category_id, publisher_id, price, copies,
available_copies, pub_year) VALUES
(1, '9788100000000', 'Database System Concepts', 1, 2, 350, 3, 3, 2005),
(2, '9788100007919', 'Fundamentals of Database Systems', 1, 3, 487, 4, 4, 2006),
(3, '9788100015838', 'Database Management Systems', 1, 4, 624, 5, 5, 2007),
(4, '9788100023757', 'A First Course in Database Systems', 1, 5, 761, 6, 6, 2008),
(5, '9788100031676', 'An Introduction to Database Systems', 1, 6, 898, 3, 3, 2009),
(6, '9788100039595', 'Database Systems: The Complete Book', 1, 7, 385, 4, 4, 2010),
(7, '9788100047514', 'The C Programming Language', 2, 8, 522, 5, 5, 2011),
(8, '9788100055433', 'Programming in ANSI C', 2, 9, 659, 6, 6, 2012),
(9, '9788100063352', 'Java: The Complete Reference', 2, 10, 796, 3, 3, 2013),
(10, '9788100071271', 'Effective Java', 2, 11, 933, 4, 4, 2014),
(11, '9788100079190', 'Head First Java', 2, 12, 420, 5, 5, 2015),
(12, '9788100087109', 'Computer Networks', 3, 13, 557, 6, 6, 2016),
(13, '9788100095028', 'Data Communications and Networking', 3, 14, 694, 3, 3, 2017),
(14, '9788100102947', 'Cryptography and Network Security', 3, 15, 831, 4, 4, 2018),
(15, '9788100110866', 'Introduction to Algorithms', 4, 16, 968, 5, 5, 2019),
(16, '9788100118785', 'The Art of Computer Programming', 4, 17, 455, 6, 6, 2020),
(17, '9788100126704', 'Algorithms', 4, 18, 592, 3, 3, 2021),
(18, '9788100134623', 'Operating Systems Concepts', 5, 19, 729, 4, 4, 2022),
(19, '9788100142542', 'Modern Operating Systems', 5, 20, 866, 5, 5, 2023),
(20, '9788100150461', 'Design Patterns', 6, 21, 353, 6, 6, 2024),
(21, '9788100158380', 'Clean Code', 6, 22, 490, 3, 3, 2005),
(22, '9788100166299', 'Software Engineering', 6, 23, 627, 4, 4, 2006),
(23, '9788100174218', 'Compilers: Principles and Tools', 4, 24, 764, 5, 5, 2007),
(24, '9788100182137', 'Data Structures Using C', 2, 25, 901, 6, 6, 2008),
(25, '9788100190056', 'Artificial Intelligence: A Modern Approach', 4, 1, 388, 3, 3,
2009);
INSERT INTO book_author (book_id, author_id) VALUES
(1, 1),
(2, 4),
(3, 6),
```

```
(4, 8),
(5, 9),
(6, 10),
(7, 12),
(8, 20),
(9, 21),
(10, 22),
(11, 23),
(12, 14),
(13, 19),
(14, 18),
(15, 15),
(16, 16),
(17, 17),
(18, 1),
(19, 14),
(20, 24),
(21, 25),
(22, 2),
(23, 8),
(24, 20),
(25, 8);
INSERT INTO member (member_id, name, email, phone, join_date, member_type) VALUES
(1, 'Aarav Sharma', 'aarav.sharma1@davv.ac.in', '9827010000', '2026-01-01',
'Student'),
(2, 'Vivaan Patel', 'vivaan.patel2@davv.ac.in', '9827010137', '2026-02-02',
'Student'),
(3, 'Aditya Verma', 'aditya.verma3@davv.ac.in', '9827010274', '2026-03-03',
'Student'),
(4, 'Vihaan Gupta', 'vihaan.gupta4@davv.ac.in', '9827010411', '2026-04-04',
'Student'),
(5, 'Arjun Singh', 'arjun.singh5@davv.ac.in', '9827010548', '2026-05-05', 'Student'),
(6, 'Sai Reddy', 'sai.reddy6@davv.ac.in', '9827010685', '2026-06-06', 'Student'),
(7, 'Reyansh Joshi', 'reyansh.joshi7@davv.ac.in', '9827010822', '2026-07-07',
'Student'),
(8, 'Krishna Mehta', 'krishna.mehta8@davv.ac.in', '9827010959', '2026-08-08',
'Student'),
(9, 'Ishaan Kulkarni', 'ishaan.kulkarni9@davv.ac.in', '9827011096', '2026-09-09',
'Student'),
(10, 'Rohan Deshmukh', 'rohan.deshmukh10@davv.ac.in', '9827011233', '2026-01-10',
'Student'),
(11, 'Ananya Iyer', 'ananya.iyer11@davv.ac.in', '9827011370', '2026-02-11',
'Student'),
(12, 'Diya Nair', 'diya.nair12@davv.ac.in', '9827011507', '2026-03-12', 'Student'),
(13, 'Saanvi Rao', 'saanvi.rao13@davv.ac.in', '9827011644', '2026-04-13', 'Student'),
(14, 'Aadhya Mishra', 'aadhya.mishra14@davv.ac.in', '9827011781', '2026-05-14',
'Student'),
(15, 'Myra Bhatt', 'myra.bhatt15@davv.ac.in', '9827011918', '2026-06-15', 'Student'),
(16, 'Kavya Jain', 'kavya.jain16@davv.ac.in', '9827012055', '2026-07-16', 'Student'),
(17, 'Riya Agarwal', 'riya.agarwal17@davv.ac.in', '9827012192', '2026-08-17',
'Student'),
(18, 'Pooja Yadav', 'pooja.yadav18@davv.ac.in', '9827012329', '2026-09-18',
'Student'),
(19, 'Neha Tiwari', 'neha.tiwari19@davv.ac.in', '9827012466', '2026-01-19',
'Student'),
(20, 'Sneha Pawar', 'sneha.pawar20@davv.ac.in', '9827012603', '2026-02-20',
'Student'),
(21, 'Dr. Rajesh Kumar', 'rajesh.kumar21@davv.ac.in', '9827012740', '2026-03-21',
'Faculty'),
(22, 'Dr. Meena Shah', 'meena.shah22@davv.ac.in', '9827012877', '2026-04-22',
'Faculty'),
(23, 'Prof. Anil Kapoor', 'anil.kapoor23@davv.ac.in', '9827013014', '2026-05-23',
'Faculty'),
(24, 'Dr. Sunita Rao', 'sunita.rao24@davv.ac.in', '9827013151', '2026-06-24',
'Faculty'),
(25, 'Prof. Vikram Soni', 'vikram.soni25@davv.ac.in', '9827013288', '2026-07-25',
'Faculty');
INSERT INTO student (member_id, roll_no, course) VALUES
(1, '2610601', 'MCA'),
(2, '2610602', 'M.Tech'),
(3, '2610603', 'MCA'),
(4, '2610604', 'M.Tech'),
```

```
(5, '2610605', 'MCA'),
(6, '2610606', 'M.Tech'),
(7, '2610607', 'MCA'),
(8, '2610608', 'M.Tech'),
(9, '2610609', 'MCA'),
(10, '2610610', 'M.Tech'),
(11, '2610611', 'MCA'),
(12, '2610611', 'M.Tech'),
(13, '2610613', 'MCA'),
(14, '2610614', 'M.Tech'),
(15, '2610615', 'MCA'),
(16, '2610616', 'M.Tech'),
(17, '2610617', 'MCA'),
(18, '2610618', 'M.Tech'),
(19, '2610619', 'MCA'),
(20, '2610620', 'M.Tech');
INSERT INTO faculty (member_id, department) VALUES
(21, 'Computer Science'),
(22, 'Information Technology'),
(23, 'Mathematics'),
(24, 'Electronics'),
(25, 'Physics');
INSERT INTO librarian (librarian_id, name, shift, salary) VALUES
(1, 'Mr. Chetan Awasthi', 'Morning', 32000),
(2, 'Mrs. Rekha Soni', 'Morning', 28000),
(3, 'Mr. Manoj Tiwari', 'Evening', 26000),
(4, 'Ms. Pallavi Joshi', 'Evening', 27000),
(5, 'Mr. Sandeep Pawar', 'Morning', 30000);
INSERT INTO issue_record (issue_id, member_id, book_id, librarian_id, issue_date,
due_date, return_date) VALUES
(1, 1, 1, 1, '2026-01-01', '2026-01-15', '2026-01-16'),
(2, 3, 4, 2, '2026-02-04', '2026-02-18', '2026-02-14'),
(3, 5, 7, 3, '2026-03-07', '2026-03-21', '2026-03-17'),
(4, 7, 10, 4, '2026-04-10', '2026-04-24', '2026-04-20'),
(5, 9, 13, 5, '2026-05-13', '2026-05-27', '2026-05-23'),
(6, 11, 16, 1, '2026-06-16', '2026-06-30', '2026-07-02'),
(7, 13, 19, 2, '2026-07-19', '2026-08-02', '2026-07-19'),
(8, 15, 22, 3, '2026-08-22', '2026-09-05', NULL),
(9, 17, 25, 4, '2026-09-25', '2026-10-09', '2026-10-05'),
(10, 19, 3, 5, '2026-01-01', '2026-01-15', '2026-01-11'),
(11, 21, 6, 1, '2026-02-04', '2026-02-18', '2026-02-21'),
(12, 23, 9, 2, '2026-03-07', '2026-03-21', '2026-03-17'),
(13, 25, 12, 3, '2026-04-10', '2026-04-24', '2026-04-20'),
(14, 2, 15, 4, '2026-05-13', '2026-05-27', '2026-05-13'),
(15, 4, 18, 5, '2026-06-16', '2026-06-30', '2026-06-26'),
(16, 6, 21, 1, '2026-07-19', '2026-08-02', NULL),
(17, 8, 24, 2, '2026-08-22', '2026-09-05', '2026-09-01'),
(18, 10, 2, 3, '2026-09-25', '2026-10-09', '2026-10-05'),
(19, 12, 5, 4, '2026-01-01', '2026-01-15', '2026-01-11'),
(20, 14, 8, 5, '2026-02-04', '2026-02-18', '2026-02-14'),
(21, 16, 11, 1, '2026-03-07', '2026-03-21', '2026-03-12'),
(22, 18, 14, 2, '2026-04-10', '2026-04-24', '2026-04-20'),
(23, 20, 17, 3, '2026-05-13', '2026-05-27', '2026-05-23'),
(24, 22, 20, 4, '2026-06-16', '2026-06-30', NULL),
(25, 24, 23, 5, '2026-07-19', '2026-08-02', '2026-07-29');
INSERT INTO fine (issue_id, fine_no, amount, paid_status) VALUES
(1, 1, 10, 'Unpaid'),
(6, 1, 12, 'Paid'),
(11, 1, 14, 'Unpaid'),
(21, 1, 10, 'Unpaid');

-- Verification
SELECT 'publisher' AS tbl, COUNT(*) AS rows_ FROM publisher UNION ALL
SELECT 'author', COUNT(*) FROM author UNION ALL
SELECT 'book', COUNT(*) FROM book UNION ALL
SELECT 'member', COUNT(*) FROM member UNION ALL
SELECT 'issue_record', COUNT(*) FROM issue_record;
```

Output:

```
tbl           rows_
publisher     25
```

```
author          25
book            25
member          25
issue_record    25
(Other tables: category 6, book_author 25, student 20, faculty 5, librarian 5, fine 4)
```

Result:

The Digital Library database was created with all constraints and each major table was loaded with 25 records.

Conclusion:

Constraints defined at the table level keep the data correct, and the populated tables provide the data for the SQL experiments.

Experiment No. 5

SQL Query Implementation

Aim:

To implement SQL queries using selection, projection, joins, nested queries, aggregate functions, GROUP BY, HAVING, set operations, views, derived relations, update and delete.

Software / Hardware Required:

MySQL Server 8.0, MySQL Workbench / MySQL command-line client; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

SQL (Structured Query Language) is the standard language of relational databases. SELECT with WHERE performs selection of rows and the column list performs projection. Joins combine rows of two or more tables on a condition (INNER, LEFT). A nested query (subquery) is a query inside another query. Aggregate functions (COUNT, SUM, AVG, MIN, MAX) work on groups of rows formed by GROUP BY, and HAVING filters the groups. UNION, INTERSECT and EXCEPT are set operations. A view is a stored query that behaves like a virtual table, and a derived relation is a subquery used in the FROM clause. UPDATE modifies and DELETE removes rows.

Procedure / Algorithm:

1. Open the digital_library database created in Experiment 4.
2. Write queries for selection, projection and the join operations.
3. Write nested queries and aggregate queries with GROUP BY and HAVING.
4. Write the set operations, create a view and use a derived relation.
5. Execute UPDATE and DELETE statements and observe the number of affected rows.

Code / Program:

```
USE digital_library;
-- 1. Selection
SELECT * FROM book WHERE price > 700;
-- 2. Projection
SELECT title, price FROM book ORDER BY price DESC;
-- 3. Join operations
SELECT m.name, b.title, i.issue_date
FROM issue_record i
JOIN member m ON m.member_id = i.member_id
JOIN book b   ON b.book_id   = i.book_id;
SELECT b.title, p.name AS publisher
FROM book b LEFT JOIN publisher p ON p.publisher_id = b.publisher_id;
-- 4. Nested queries
SELECT title FROM book
WHERE book_id IN (SELECT book_id FROM issue_record WHERE return_date IS NULL);
SELECT title, price FROM book WHERE price > (SELECT AVG(price) FROM book);
-- 5. Aggregate functions
SELECT COUNT(*) AS total_books, SUM(copies) AS total_copies, AVG(price) AS avg_price,
       MIN(price) AS min_price, MAX(price) AS max_price FROM book;
-- 6. GROUP BY and 7. HAVING
SELECT c.cat_name, COUNT(*) AS books, AVG(b.price) AS avg_price
FROM book b JOIN category c ON c.category_id = b.category_id
GROUP BY c.cat_name
HAVING COUNT(*) >= 4;
-- 8. Set operations (INTERSECT / EXCEPT need MySQL 8.0.31+)
SELECT member_id FROM issue_record WHERE book_id <= 10
UNION
SELECT member_id FROM issue_record WHERE book_id > 20;
SELECT member_id FROM issue_record WHERE book_id <= 15
INTERSECT
SELECT member_id FROM issue_record WHERE book_id > 10;
SELECT member_id FROM issue_record WHERE book_id <= 15
EXCEPT
SELECT member_id FROM issue_record WHERE book_id > 15;
-- 9. Views
```

```
CREATE VIEW pending_returns AS
SELECT i.issue_id, m.name AS member_name, b.title, i.due_date
FROM issue_record i
JOIN member m ON m.member_id = i.member_id
JOIN book b   ON b.book_id   = i.book_id
WHERE i.return_date IS NULL;
SELECT * FROM pending_returns;
-- 10. Derived relations
SELECT t.cat_name, t.books
FROM (SELECT c.cat_name, COUNT(*) AS books
      FROM book b JOIN category c ON c.category_id = b.category_id
      GROUP BY c.cat_name) AS t
WHERE t.books > 3;
-- 11. Update
UPDATE book SET price = price * 1.10 WHERE category_id = 1;
-- 12. Delete
DELETE FROM fine WHERE paid_status = 'Paid';
```

Output:

```
1. Selection        : 10 books with price > 700
3. Join             : 25 rows (member, book, issue date)
4. Nested query     : Design Patterns, Clean Code, Software Engineering (not yet
returned)
5. Aggregate        : total_books 25, total_copies 111, avg_price 642.00, min 350, max
968
6. GROUP BY/HAVING  : Algorithms 5 (633.40), Database 6 (584.17), Programming 6
(705.17)
8. Set operations   : UNION 15 rows, INTERSECT 5 rows, EXCEPT 15 rows
9. View             : pending_returns shows 3 rows
11. Update          : 6 rows affected (Database books price +10%)
12. Delete          : 1 row affected (paid fine removed)
```

Result:

All the SQL queries were executed successfully and the expected results were obtained.

Conclusion:

SQL provides powerful operations for retrieving, summarising and modifying the data of a relational database.

Experiment No. 6

Relational Algebra

Aim:

To write relational algebra expressions for selection, projection, union, difference, Cartesian product, join and division on the Digital Library database.

Software / Hardware Required:

Digital Library case study; MS Word / text editor for the SRS, draw.io or Graphviz for diagrams; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Relational algebra is a procedural query language whose operations take one or two relations and produce a relation. Selection σ picks rows that satisfy a condition, projection π keeps the listed columns, union ∪ combines the tuples of two union-compatible relations, difference − gives the tuples present in the first but not in the second, Cartesian product × pairs every tuple of one relation with every tuple of the other, and join ⋈ is a product followed by a selection on the common attribute. Division ÷ finds the values of A that are related to all the values of B, and is used for 'for all' queries.

Procedure / Algorithm:

1. Write the schema of the relations used (Book, Member, Issue_Record, Publisher).
2. Write the expression for selection and projection.
3. Write the expressions for union and difference on union-compatible projections.
4. Write the Cartesian product and the join (natural and theta join).
5. Write the division expression and give the equivalent SQL for each; verify the results in MySQL.

Relational Algebra Expressions:

Relations: Book(book_id, title, category_id, publisher_id, price, ...), Publisher(publisher_id, name, city), Issue_Record(issue_id, member_id, book_id, librarian_id, ...)

No.	Operation	Relational algebra expression	Equivalent SQL
1	Selection	σ price>700 (Book)	SELECT * FROM book WHERE price > 700;
2	Projection	π title, price (Book)	SELECT title, price FROM book;
3	Union	π member_id (σ book_id<=10 (Issue_Record)) ∪ π member_id (σ book_id>20 (Issue_Record))	SELECT member_id FROM issue_record WHERE book_id <= 10 UNION SELECT member_id FROM issue_record WHERE book_id > 20;
4	Difference	π member_id (σ book_id<=15 (Issue_Record)) − π member_id (σ book_id>15 (Issue_Record))	... EXCEPT ... (members who issued only books with id up to 15)
5	Cartesian product	Book × Publisher	SELECT * FROM book CROSS JOIN publisher; (25 x 25 = 625 tuples)
6	Natural join	Book ⋈ Publisher	SELECT * FROM book NATURAL JOIN publisher;
7	Theta join with projection	π title, name (Book ⋈ Book.publisher_id = Publisher.publisher_id Publisher)	SELECT b.title, p.name FROM book b JOIN publisher p ON b.publisher_id = p.publisher_id;
8	Division	π member_id, book_id (Issue_Record) ÷ π book_id (σ category_id=5 (Book))	SELECT DISTINCT member_id FROM issue_record i1 WHERE NOT EXISTS (SELECT book_id FROM book WHERE category_id = 5 AND book_id NOT IN (SELECT book_id FROM issue_record i2 WHERE i2.member_id = i1.member_id));

Small example of division

R(member, book) = {(1,A), (1,B), (2,A), (3,A), (3,B)} and S(book) = {A, B}. Then R ÷ S = {1, 3}, because only members 1 and 3 are related to both A and B.

Output:

```
Selection : 10 tuples        Projection : 25 tuples        Union : 15 tuples
Difference: 15 tuples (member ids 1,2,3,5,7,9,10,12,14,16,18,19,21,23,25)
Product   : 625 tuples        Join       : 25 tuples
Division  : empty set on the sample data (no member has issued both Operating Systems
books); the small example above gives {1, 3}
```

Result:

Relational algebra expressions for all seven operations were written and verified with the equivalent SQL queries.

Conclusion:

Relational algebra forms the theoretical basis of SQL, and each algebra operation can be expressed by an SQL query.

Experiment No. 7

Advanced SQL Programming

Aim:

To develop stored procedures, functions, triggers, views and transactions using PL/SQL on Oracle for the Digital Library database.

Software / Hardware Required:

Oracle Database 19c / 21c XE, SQL*Plus or Oracle SQL Developer; PC with minimum 8 GB RAM, Windows / Linux.

Theory:

PL/SQL is Oracle's procedural extension of SQL; a program is written in blocks (DECLARE, BEGIN, EXCEPTION, END). A stored procedure performs an action and a function returns a value; both are stored in the database and can be called repeatedly. A cursor processes the rows of a query one at a time. A package groups related procedures and functions into a specification and a body. A trigger runs automatically when an INSERT, UPDATE or DELETE happens. A view is a stored query, and a transaction (COMMIT / ROLLBACK) makes a group of statements atomic.

Procedure / Algorithm:

1. Create the tables of Experiment 4 in Oracle (use VARCHAR2 / NUMBER) and insert the sample rows shown in the script.
2. Create the view v_book_availability.
3. Create the function calc_fine and the procedure issue_book with a transaction and exception handling.
4. Create the package pkg_library with a cursor-based procedure for overdue books.
5. Create the trigger trg_default_due and test everything with an anonymous PL/SQL block.

Code / Program:

```
-- Sample data (Oracle); tables created as in Experiment 4
INSERT INTO category VALUES (1, 'Database');
INSERT INTO publisher VALUES (1, 'McGraw-Hill', 'New Delhi', 'info@mcgrawhill.com');
INSERT INTO book (book_id, isbn, title, category_id, publisher_id, price, copies,
available_copies, pub_year)
  VALUES (1, '9788100000001', 'Database System Concepts', 1, 1, 750, 3, 3, 2019);
INSERT INTO member (member_id, name, email, phone, join_date, member_type)
  VALUES (1, 'Aarav Sharma', 'aarav.sharma@davv.ac.in', '9827010000', SYSDATE,
'Student');
INSERT INTO librarian (librarian_id, name, shift, salary) VALUES (1, 'Chetan Awasthi',
'Morning', 32000);
COMMIT;

-- 1. View
CREATE OR REPLACE VIEW v_book_availability AS
SELECT book_id, title, available_copies,
       CASE WHEN available_copies > 0 THEN 'Available' ELSE 'Not Available' END AS
stock_status
FROM book;

-- 2. Trigger: set due date automatically (14 days)
CREATE OR REPLACE TRIGGER trg_default_due
BEFORE INSERT ON issue_record FOR EACH ROW
BEGIN
  IF :NEW.due_date IS NULL THEN
    :NEW.due_date := :NEW.issue_date + 14;
  END IF;
END;
/
-- 3. Function: fine at Rs. 2 per late day
CREATE OR REPLACE FUNCTION calc_fine(p_issue_id NUMBER) RETURN NUMBER IS
  v_due DATE; v_ret DATE; v_days NUMBER;
BEGIN
  SELECT due_date, NVL(return_date, SYSDATE) INTO v_due, v_ret
```

```
  FROM issue_record WHERE issue_id = p_issue_id;
  v_days := TRUNC(v_ret) - TRUNC(v_due);
  IF v_days > 0 THEN RETURN v_days * 2; ELSE RETURN 0; END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN RETURN -1;
END;
/
-- 4. Procedure with transaction and exception handling
CREATE OR REPLACE PROCEDURE issue_book(p_issue NUMBER, p_member NUMBER, p_book NUMBER,
p_lib NUMBER) IS
  v_avail NUMBER;
BEGIN
  SELECT available_copies INTO v_avail FROM book WHERE book_id = p_book FOR UPDATE;
  IF v_avail <= 0 THEN
    RAISE_APPLICATION_ERROR(-20001, 'No copy available');
  END IF;
  INSERT INTO issue_record (issue_id, member_id, book_id, librarian_id, issue_date)
  VALUES (p_issue, p_member, p_book, p_lib, SYSDATE);
  UPDATE book SET available_copies = available_copies - 1 WHERE book_id = p_book;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Book issued. Issue id = ' || p_issue);
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Issue failed: ' || SQLERRM);
END;
/
-- 5. Package with a cursor
CREATE OR REPLACE PACKAGE pkg_library IS
  PROCEDURE show_overdue;
  FUNCTION issued_count(p_member NUMBER) RETURN NUMBER;
END pkg_library;
/
CREATE OR REPLACE PACKAGE BODY pkg_library IS
  PROCEDURE show_overdue IS
    CURSOR c IS SELECT i.issue_id, m.name, b.title, i.due_date
                FROM issue_record i JOIN member m ON m.member_id = i.member_id
                JOIN book b ON b.book_id = i.book_id
                WHERE i.return_date IS NULL AND i.due_date < SYSDATE;
  BEGIN
    FOR r IN c LOOP
      DBMS_OUTPUT.PUT_LINE(r.issue_id || ' ' || r.name || ' ' || r.title || ' due ' ||
r.due_date);
    END LOOP;
  END;
  FUNCTION issued_count(p_member NUMBER) RETURN NUMBER IS v NUMBER;
  BEGIN
    SELECT COUNT(*) INTO v FROM issue_record WHERE member_id = p_member AND
return_date IS NULL;
    RETURN v;
  END;
END pkg_library;
/
-- Testing
SET SERVEROUTPUT ON
EXEC issue_book(101, 1, 1, 1);
EXEC issue_book(102, 1, 1, 1);
SELECT * FROM v_book_availability;
SELECT calc_fine(101) AS fine FROM dual;
SELECT pkg_library.issued_count(1) AS issued FROM dual;
EXEC pkg_library.show_overdue;
```

Output:

```
Book issued. Issue id = 101
Book issued. Issue id = 102


BOOK_ID TITLE                     AVAILABLE_COPIES STOCK_STATUS
      1 Database System Concepts                1 Available


FINE
----
```

```
   0
ISSUED
------
     2
(No overdue books, so show_overdue prints nothing. A failed issue prints 'Issue
failed: ORA-20001: No copy available'.)
```

Result:

The view, function, procedure, package with cursor and trigger were created and tested successfully in Oracle.

Conclusion:

PL/SQL stored programs keep the business logic inside the database and make operations faster, reusable and consistent.

Experiment No. 8

Database Integrity and Triggers

Aim:

To implement domain, entity and referential integrity, CHECK constraints and triggers for audit logging, automatic stock update, attendance calculation and salary update, and to test them with valid and invalid inputs.

Software / Hardware Required:

Oracle Database 19c / 21c XE, SQL*Plus or Oracle SQL Developer; PC with minimum 8 GB RAM, Windows / Linux.

Theory:

Integrity constraints keep the data correct. Domain constraints restrict the values of a column (data type, CHECK), entity integrity requires a unique, non-null primary key, and referential integrity requires every foreign key value to exist in the parent table. A trigger is a PL/SQL block that fires automatically before or after an INSERT, UPDATE or DELETE; in a row-level trigger the old and new values are available as :OLD and :NEW. Triggers are used for auditing, automatic updates of dependent data and for enforcing complex rules. RAISE_APPLICATION_ERROR rejects an invalid operation.

Procedure / Algorithm:

1. Use the Oracle tables of Experiment 7 and create audit_log, book_purchase and staff_attendance tables.
2. Test domain, entity, referential and CHECK constraints with valid and invalid INSERT statements.
3. Create the audit trigger on the price of Book.
4. Create the stock trigger on Book_Purchase, the attendance trigger on Staff_Attendance and the salary trigger on Librarian.
5. Test each trigger with valid and invalid data and observe the results.

Code / Program:

```
-- Supporting tables
ALTER TABLE librarian ADD days_present NUMBER DEFAULT 0;
CREATE TABLE audit_log (
  log_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  table_name VARCHAR2(30), action VARCHAR2(10),
  old_value VARCHAR2(100), new_value VARCHAR2(100),
  changed_by VARCHAR2(30), changed_on DATE);
CREATE TABLE book_purchase (
  purchase_id NUMBER PRIMARY KEY, book_id NUMBER REFERENCES book(book_id), qty NUMBER
CHECK (qty > 0));
CREATE TABLE staff_attendance (
  att_id NUMBER PRIMARY KEY, librarian_id NUMBER REFERENCES librarian(librarian_id),
  att_date DATE, status CHAR(1) CHECK (status IN ('P','A')));

-- PART A: constraint tests
INSERT INTO book
(book_id,isbn,title,category_id,publisher_id,price,copies,available_copies)
  VALUES (2,'9788100000002','Valid Book',1,1,500,2,2);                    -- valid
INSERT INTO book
(book_id,isbn,title,category_id,publisher_id,price,copies,available_copies)
  VALUES (3,'9788100000003','Bad Price',1,1,-100,2,2);                    -- domain /
CHECK
INSERT INTO book
(book_id,isbn,title,category_id,publisher_id,price,copies,available_copies)
  VALUES (1,'9788100000009','Duplicate Id',1,1,300,1,1);                  -- entity
integrity
INSERT INTO book
(book_id,isbn,title,category_id,publisher_id,price,copies,available_copies)
  VALUES (NULL,'9788100000010','Null Id',1,1,300,1,1);                    -- entity
integrity
INSERT INTO issue_record (issue_id,member_id,book_id,librarian_id,issue_date,due_date)
```

```
  VALUES (301,999,1,1,SYSDATE,SYSDATE+14);                                    --
referential integrity
INSERT INTO issue_record (issue_id,member_id,book_id,librarian_id,issue_date,due_date)
  VALUES (302,1,1,1,SYSDATE,SYSDATE-5);                                       -- CHECK
(due_date)
DELETE FROM member WHERE member_id = 1;                                       -- child
rows exist

-- PART B: triggers
CREATE OR REPLACE TRIGGER trg_book_audit            -- audit logging
AFTER UPDATE OF price ON book FOR EACH ROW
BEGIN
  INSERT INTO audit_log (table_name, action, old_value, new_value, changed_by,
changed_on)
  VALUES ('BOOK', 'UPDATE', :OLD.price, :NEW.price, USER, SYSDATE);
END;
/
CREATE OR REPLACE TRIGGER trg_stock_update          -- automatic stock update
AFTER INSERT ON book_purchase FOR EACH ROW
BEGIN
  UPDATE book SET copies = copies + :NEW.qty,
                  available_copies = available_copies + :NEW.qty
  WHERE book_id = :NEW.book_id;
END;
/
CREATE OR REPLACE TRIGGER trg_attendance            -- attendance calculation
AFTER INSERT ON staff_attendance FOR EACH ROW WHEN (NEW.status = 'P')
BEGIN
  UPDATE librarian SET days_present = NVL(days_present, 0) + 1
  WHERE librarian_id = :NEW.librarian_id;
END;
/
CREATE OR REPLACE TRIGGER trg_salary_update         -- salary update rule
BEFORE UPDATE OF salary ON librarian FOR EACH ROW
BEGIN
  IF :NEW.salary < :OLD.salary THEN
    RAISE_APPLICATION_ERROR(-20002, 'Salary cannot be reduced');
  END IF;
  INSERT INTO audit_log (table_name, action, old_value, new_value, changed_by,
changed_on)
  VALUES ('LIBRARIAN', 'UPDATE', :OLD.salary, :NEW.salary, USER, SYSDATE);
END;
/
-- Tests
UPDATE book SET price = 800 WHERE book_id = 1;                          -- audit row
created
INSERT INTO book_purchase VALUES (1, 1, 5);                             -- copies 3 -> 8
INSERT INTO staff_attendance VALUES (1, 1, SYSDATE, 'P');               -- days_present
1
INSERT INTO staff_attendance VALUES (2, 1, SYSDATE+1, 'A');             -- no change
UPDATE librarian SET salary = 35000 WHERE librarian_id = 1;             -- valid
UPDATE librarian SET salary = 20000 WHERE librarian_id = 1;             -- rejected
SELECT * FROM audit_log;
SELECT book_id, copies, available_copies FROM book WHERE book_id = 1;
SELECT librarian_id, days_present FROM librarian;
```

Output:

```
PART A
  valid book insert        : 1 row created
  price = -100             : ORA-02290: check constraint violated
  duplicate book_id        : ORA-00001: unique constraint violated
  NULL book_id             : ORA-01400: cannot insert NULL into BOOK_ID
  member_id 999            : ORA-02291: integrity constraint violated - parent key not
found
  due_date < issue_date    : ORA-02290: check constraint violated
  delete member 1          : ORA-02292: integrity constraint violated - child record
found
PART B
  audit_log                : BOOK 750 -> 800 and LIBRARIAN 32000 -> 35000
  book 1 after purchase    : copies 8, available_copies 6
  librarian days_present   : 1
```

```
salary = 20000              : ORA-20002: Salary cannot be reduced
```

Result:

The integrity constraints and the four triggers worked correctly for valid inputs and rejected the invalid inputs.

Conclusion:

Constraints and triggers protect the database from invalid data and automate dependent updates without changing the application code.

Experiment No. 9

Transaction Processing

Aim:

To demonstrate COMMIT, ROLLBACK, SAVEPOINT and nested transactions and to document the effect of each operation.

Software / Hardware Required:

Oracle Database 19c / 21c XE, SQL*Plus or Oracle SQL Developer; PC with minimum 8 GB RAM, Windows / Linux.

Theory:

A transaction is a logical unit of work that must be atomic, consistent, isolated and durable (ACID). COMMIT makes all the changes of the transaction permanent. ROLLBACK undoes all uncommitted changes. SAVEPOINT marks a point inside the transaction so that ROLLBACK TO savepoint undoes only the work after that point. Oracle does not support true nested transactions, but a nested (independent) transaction can be obtained with an autonomous transaction (PRAGMA AUTONOMOUS_TRANSACTION), which commits or rolls back separately from the calling transaction.

Procedure / Algorithm:

1. Note the initial state of the book and member tables.
2. Execute DML statements and use SAVEPOINT after each step.
3. Use ROLLBACK TO a savepoint and observe which changes are undone.
4. Use COMMIT and then ROLLBACK to show that committed data is permanent.
5. Create an autonomous-transaction procedure and call it from an outer transaction that is rolled back.

Code / Program:

```
-- Initial state
SELECT available_copies FROM book WHERE book_id = 1;          -- e.g. 6
SELECT status FROM member WHERE member_id = 1;                -- Active

-- 1. SAVEPOINT and ROLLBACK TO
UPDATE book SET available_copies = available_copies - 1 WHERE book_id = 1;
SAVEPOINT sp1;
INSERT INTO issue_record (issue_id, member_id, book_id, librarian_id, issue_date,
due_date)
VALUES (401, 1, 1, 1, SYSDATE, SYSDATE + 14);
SAVEPOINT sp2;
UPDATE member SET status = 'Suspended' WHERE member_id = 1;
ROLLBACK TO sp2;                                              -- only the suspend is
undone
SELECT status FROM member WHERE member_id = 1;                -- Active
ROLLBACK TO sp1;                                              -- the insert is undone
SELECT COUNT(*) FROM issue_record WHERE issue_id = 401;       -- 0
SELECT available_copies FROM book WHERE book_id = 1;          -- 5 (update still
pending)

-- 2. COMMIT
COMMIT;                                                       -- update of book is
permanent

-- 3. ROLLBACK
INSERT INTO issue_record (issue_id, member_id, book_id, librarian_id, issue_date,
due_date)
VALUES (402, 1, 1, 1, SYSDATE, SYSDATE + 14);
ROLLBACK;                                                     -- complete transaction
undone
SELECT COUNT(*) FROM issue_record WHERE issue_id = 402;       -- 0
SELECT available_copies FROM book WHERE book_id = 1;          -- 5 (committed value
remains)

-- 4. Nested (autonomous) transaction
```

```
CREATE OR REPLACE PROCEDURE log_event(p_msg VARCHAR2) IS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO audit_log (table_name, action, new_value, changed_by, changed_on)
  VALUES ('EVENT', 'LOG', p_msg, USER, SYSDATE);
  COMMIT;                                                    -- commits only this
inner work
END;
/
BEGIN
  UPDATE book SET available_copies = available_copies - 1 WHERE book_id = 1;   --
outer work
  log_event('Outer transaction started');                                      --
inner, committed
  ROLLBACK;                                                                    --
outer undone
END;
/
SELECT available_copies FROM book WHERE book_id = 1;                           -- 5
SELECT new_value FROM audit_log WHERE table_name = 'EVENT';                    -- row
remains
```

Output:

```
Operation                          Effect
UPDATE + SAVEPOINT sp1             book copies 6 -> 5 (not yet permanent)
INSERT + SAVEPOINT sp2             issue 401 inserted
ROLLBACK TO sp2                    member status back to Active; insert 401 still
present
ROLLBACK TO sp1                    issue 401 removed; book update still pending
COMMIT                             book copies = 5 saved permanently
INSERT 402 + ROLLBACK              issue 402 removed; copies remain 5
Autonomous procedure + ROLLBACK    outer update undone (copies 5); audit row 'Outer
transaction started' remains
```

Result:

COMMIT, ROLLBACK, SAVEPOINT and the autonomous (nested) transaction were demonstrated and the effect of each was documented.

Conclusion:

Transaction control statements keep the database consistent: committed work is permanent, while rollback and savepoints undo unwanted changes.

Experiment No. 10

MongoDB CRUD Operations

Aim:

To create a MongoDB database for the Digital Library and to perform insert, update, delete, find, sort and projection operations on a collection of at least 20 documents.

Software / Hardware Required:

MongoDB Community Server 7.x, mongosh (MongoDB Shell) or MongoDB Compass; PC with minimum 4 GB RAM.

Theory:

MongoDB is a document-oriented NoSQL database that stores data as flexible JSON-like (BSON) documents in collections; a collection need not have a fixed schema. CRUD operations are: insertOne / insertMany (create), find (read), updateOne / updateMany with operators such as $set and $inc (update) and deleteOne / deleteMany (delete). find() takes a filter and an optional projection, sort() orders the result (1 ascending, -1 descending), and operators such as $gt, $in and $regex are used in filters.

Procedure / Algorithm:

1. Start mongosh and select the database with use digital_library.
2. Insert 20 book documents into the books collection using insertMany().
3. Read the documents using find() with filters, projection and sort().
4. Update documents with updateOne() and updateMany().
5. Delete documents with deleteOne() and deleteMany() and verify with countDocuments().

Code / Program:

```
use digital_library

// Insert (20 documents)
db.books.insertMany([
 {_id:1,  title:"Database System Concepts",  authors:["Silberschatz","Korth"],
category:"Database",    price:750, copies:5, year:2019},
 {_id:2,  title:"Fundamentals of Database Systems", authors:["Elmasri","Navathe"],
category:"Database", price:820, copies:4, year:2016},
 {_id:3,  title:"Database Management Systems", authors:["Ramakrishnan","Gehrke"],
category:"Database", price:690, copies:3, year:2014},
 {_id:4,  title:"The C Programming Language", authors:["Kernighan","Ritchie"],
category:"Programming", price:450, copies:6, year:1988},
 {_id:5,  title:"Java: The Complete Reference", authors:["Herbert Schildt"],
category:"Programming", price:796, copies:4, year:2022},
 {_id:6,  title:"Effective Java", authors:["Joshua Bloch"], category:"Programming",
price:933, copies:2, year:2018},
 {_id:7,  title:"Head First Java", authors:["Kathy Sierra"], category:"Programming",
price:560, copies:5, year:2005},
 {_id:8,  title:"Computer Networks", authors:["Tanenbaum"], category:"Networking",
price:640, copies:4, year:2010},
 {_id:9,  title:"Data Communications and Networking", authors:["Forouzan"],
category:"Networking", price:710, copies:3, year:2012},
 {_id:10, title:"Cryptography and Network Security", authors:["Stallings"],
category:"Networking", price:680, copies:3, year:2017},
 {_id:11, title:"Introduction to Algorithms", authors:["Cormen","Leiserson"],
category:"Algorithms", price:968, copies:5, year:2009},
 {_id:12, title:"The Art of Computer Programming", authors:["Knuth"],
category:"Algorithms", price:950, copies:2, year:2011},
 {_id:13, title:"Algorithms", authors:["Sedgewick","Wayne"], category:"Algorithms",
price:620, copies:4, year:2011},
 {_id:14, title:"Operating System Concepts", authors:["Silberschatz","Galvin"],
category:"Operating Systems", price:700, copies:5, year:2018},
 {_id:15, title:"Modern Operating Systems", authors:["Tanenbaum"], category:"Operating
Systems", price:650, copies:3, year:2014},
 {_id:16, title:"Design Patterns", authors:["Gamma","Helm"], category:"Software
Engineering", price:540, copies:3, year:1994},
```

```
 {_id:17, title:"Clean Code", authors:["Robert Martin"], category:"Software
Engineering", price:520, copies:4, year:2008},
 {_id:18, title:"Software Engineering", authors:["Sommerville"], category:"Software
Engineering", price:590, copies:5, year:2015},
 {_id:19, title:"Compilers: Principles and Tools", authors:["Aho","Ullman"],
category:"Algorithms", price:730, copies:2, year:2006},
 {_id:20, title:"Data Structures Using C", authors:["Balagurusamy"],
category:"Programming", price:350, copies:6, year:2013}
])

// Find
db.books.find()
db.books.find({category:"Database"})
db.books.find({price:{$gt:700}})
db.books.find({$or:[{category:"Networking"},{year:{$lt:2000}}]})

// Sort
db.books.find().sort({price:-1})
db.books.find({category:"Programming"}).sort({title:1})

// Projection (only title and price)
db.books.find({}, {_id:0, title:1, price:1})
db.books.find({category:"Database"}, {_id:0, title:1,
authors:1}).sort({year:-1}).limit(2)

// Update
db.books.updateOne({_id:1}, {$set:{price:800}})
db.books.updateMany({category:"Algorithms"}, {$inc:{copies:1}})

// Delete
db.books.deleteOne({_id:20})
db.books.deleteMany({year:{$lt:1990}})
db.books.countDocuments()
```

Output:

```
insertMany      : acknowledged: true, 20 documents inserted
find (Database) : 3 documents (_id 1, 2, 3)
price > 700     : 8 documents
sort price -1   : Introduction to Algorithms (968), Effective Java (933), ...
projection      : { title: 'Database System Concepts', price: 750 } ...
updateOne       : matchedCount 1, modifiedCount 1
updateMany      : matchedCount 4, modifiedCount 4 (copies + 1)
deleteOne       : deletedCount 1
deleteMany      : deletedCount 1 (The C Programming Language, 1988)
countDocuments  : 18
```

Result:

The MongoDB database was created with 20 documents and all CRUD, sort and projection operations were performed successfully.

Conclusion:

MongoDB stores data as flexible documents and provides simple commands for CRUD operations without a fixed schema.

Experiment No. 11

MongoDB Aggregation Framework

Aim:

To implement aggregation pipelines using $match, $group, $project, $sort, $lookup and $unwind and to generate analytical reports from the Digital Library dataset.

Software / Hardware Required:

MongoDB Community Server 7.x, mongosh (MongoDB Shell) or MongoDB Compass; PC with minimum 4 GB RAM.

Theory:

The aggregation framework processes documents through a pipeline of stages, where the output of one stage is the input of the next. $match filters documents, $group groups them and computes accumulators such as $sum, $avg and $max, $project reshapes the documents, $sort orders them, $unwind deconstructs an array field into separate documents, and $lookup performs a left outer join with another collection. Pipelines are used to generate analytical reports directly inside the database.

Procedure / Algorithm:

1. Use the books collection of Experiment 10 and create the members and issues collections.
2. Write a pipeline with $match, $group, $project and $sort to report category-wise statistics.
3. Use $unwind and $group to count books per author.
4. Use $lookup (and $unwind) to join issues with books and members.
5. Generate the reports: most issued books, member-wise issue count and category-wise average price.

Code / Program:

```
// Supporting collections
db.members.insertMany([
 {_id:1, name:"Aarav Sharma", type:"Student"}, {_id:2, name:"Vivaan Patel",
type:"Student"},
 {_id:3, name:"Ananya Iyer",  type:"Student"}, {_id:4, name:"Dr. Rajesh Kumar",
type:"Faculty"},
 {_id:5, name:"Prof. Anil Kapoor", type:"Faculty"}])
db.issues.insertMany([
 {member_id:1, book_id:1,  date:ISODate("2026-07-01")}, {member_id:1, book_id:11,
date:ISODate("2026-07-10")},
 {member_id:2, book_id:1,  date:ISODate("2026-07-12")}, {member_id:2, book_id:5,
date:ISODate("2026-07-15")},
 {member_id:3, book_id:11, date:ISODate("2026-08-02")}, {member_id:3, book_id:14,
date:ISODate("2026-08-05")},
 {member_id:4, book_id:1,  date:ISODate("2026-08-09")}, {member_id:4, book_id:16,
date:ISODate("2026-08-11")},
 {member_id:5, book_id:11, date:ISODate("2026-08-20")}, {member_id:5, book_id:2,
date:ISODate("2026-08-22")}])

// 1. $match + $group + $project + $sort : category-wise report
db.books.aggregate([
 {$match: {price: {$gt: 500}}},
 {$group: {_id: "$category", books: {$sum: 1}, avgPrice: {$avg: "$price"},
totalCopies: {$sum: "$copies"}}},
 {$project: {_id: 0, category: "$_id", books: 1, totalCopies: 1, avgPrice: {$round:
["$avgPrice", 2]}}},
 {$sort: {avgPrice: -1}}
])

// 2. $unwind : number of books per author
db.books.aggregate([
 {$unwind: "$authors"},
 {$group: {_id: "$authors", titles: {$sum: 1}}},
 {$match: {titles: {$gte: 2}}},
 {$sort: {titles: -1}}
])
```

```
// 3. $lookup : most issued books
db.issues.aggregate([
 {$group: {_id: "$book_id", issued: {$sum: 1}}},
 {$lookup: {from: "books", localField: "_id", foreignField: "_id", as: "book"}},
 {$unwind: "$book"},
 {$project: {_id: 0, title: "$book.title", issued: 1}},
 {$sort: {issued: -1}}, {$limit: 3}
])

// 4. $lookup : member-wise report
db.issues.aggregate([
 {$lookup: {from: "members", localField: "member_id", foreignField: "_id", as: "m"}},
 {$unwind: "$m"},
 {$group: {_id: "$m.name", type: {$first: "$m.type"}, books: {$sum: 1}}},
 {$sort: {books: -1, _id: 1}}
])
```

Output:

```
Report 1 (category-wise)       : Algorithms 4 books avg 817.00 | Programming 3 avg
763.00 | Database 3 avg 753.33 | Networking 3 avg 676.67 | Operating Systems 2 avg
675.00 | Software Engineering 3 avg 550.00
Report 2 (books per author)    : Silberschatz 2, Tanenbaum 2
Report 3 (most issued books)   : Database System Concepts 3, Introduction to
Algorithms 3, then a book issued once
Report 4 (member-wise)         : Aarav Sharma 2, Ananya Iyer 2, Dr. Rajesh Kumar 2,
Prof. Anil Kapoor 2, Vivaan Patel 2
```

Result:

Aggregation pipelines using all the required stages were executed and the analytical reports were generated.

Conclusion:

The aggregation framework allows complex analysis, such as grouping and joins, to be performed inside MongoDB itself.

Experiment No. 12

Graph Database using Neo4j

Aim:

To design a graph database for a Research Collaboration Network and to implement node creation, relationship creation, Cypher queries, shortest path analysis and recommendation queries using Neo4j.

Software / Hardware Required:

Neo4j Community Edition 5.x / Neo4j Desktop, Neo4j Browser (Cypher); PC with minimum 4 GB RAM.

Theory:

A graph database stores data as nodes (entities) and relationships (connections), both of which can have properties. It is suitable for highly connected data such as social and collaboration networks, since relationships are stored directly and traversals are fast. Neo4j uses the Cypher query language: CREATE builds nodes and relationships, MATCH finds patterns, WHERE filters, and RETURN gives the result. shortestPath() finds the shortest connection between two nodes, and a recommendation query finds nodes that are connected through a common neighbour but not directly connected.

Procedure / Algorithm:

1. Design the graph: nodes Author, Paper, Institution; relationships WROTE, WORKS_AT, CITES and CO_AUTHOR.
2. Create the nodes and the relationships with CREATE.
3. Derive the CO_AUTHOR relationships from authors of the same paper.
4. Write Cypher queries for papers, collaborators and citations.
5. Find the shortest path between two authors and write the recommendation query.

Code / Program:

```
// 1. Nodes
CREATE (a:Author {name:'Asha Verma',   field:'Databases'}),
       (b:Author {name:'Rahul Mehta',  field:'Databases'}),
       (c:Author {name:'Sneha Iyer',   field:'Data Mining'}),
       (d:Author {name:'Karan Joshi',  field:'NoSQL'}),
       (e:Author {name:'Meera Nair',   field:'Spatial Data'}),
       (f:Author {name:'Farhan Ali',   field:'Graph Analytics'}),
       (i1:Institution {name:'DAVV Indore'}), (i2:Institution {name:'IIT Indore'}),
       (p1:Paper {title:'Graph Indexing',     year:2021}),
       (p2:Paper {title:'Query Optimization', year:2022}),
       (p3:Paper {title:'NoSQL Benchmarks',   year:2023}),
       (p4:Paper {title:'Spatial Joins',      year:2024}),
       (p5:Paper {title:'Link Analysis',      year:2025})

// 2. Relationships
MATCH (a:Author {name:'Asha Verma'}), (b:Author {name:'Rahul Mehta'}), (c:Author
{name:'Sneha Iyer'}),
      (d:Author {name:'Karan Joshi'}), (e:Author {name:'Meera Nair'}), (f:Author
{name:'Farhan Ali'}),
      (p1:Paper {title:'Graph Indexing'}), (p2:Paper {title:'Query Optimization'}),
      (p3:Paper {title:'NoSQL Benchmarks'}), (p4:Paper {title:'Spatial Joins'}),
(p5:Paper {title:'Link Analysis'})
CREATE (a)-[:WROTE]->(p1), (b)-[:WROTE]->(p1), (b)-[:WROTE]->(p2), (c)-[:WROTE]->(p2),
       (c)-[:WROTE]->(p3), (d)-[:WROTE]->(p3), (d)-[:WROTE]->(p4), (e)-[:WROTE]->(p4),
       (e)-[:WROTE]->(p5), (f)-[:WROTE]->(p5),
       (p2)-[:CITES]->(p1), (p3)-[:CITES]->(p2), (p4)-[:CITES]->(p3), (p5)-[:CITES]-
>(p4)

MATCH (x:Author), (i:Institution {name:'DAVV Indore'}) WHERE x.name IN ['Asha
Verma','Rahul Mehta','Sneha Iyer']
CREATE (x)-[:WORKS_AT]->(i)
MATCH (x:Author), (i:Institution {name:'IIT Indore'}) WHERE x.name IN ['Karan
Joshi','Meera Nair','Farhan Ali']
CREATE (x)-[:WORKS_AT]->(i)
```

```
// Co-author relationships from common papers
MATCH (x:Author)-[:WROTE]->(:Paper)<-[:WROTE]-(y:Author) WHERE x.name < y.name
MERGE (x)-[:CO_AUTHOR]-(y)

// 3. Cypher queries
MATCH (a:Author)-[:WROTE]->(p:Paper) RETURN a.name, p.title ORDER BY p.year
MATCH (a:Author {name:'Rahul Mehta'})-[:CO_AUTHOR]-(c) RETURN c.name
MATCH (p:Paper)-[:CITES]->(q:Paper {title:'Graph Indexing'}) RETURN p.title
MATCH (a:Author)-[:WORKS_AT]->(i:Institution) RETURN i.name, count(a) AS authors

// 4. Shortest path analysis
MATCH path = shortestPath((a:Author {name:'Asha Verma'})-[:CO_AUTHOR*]-(f:Author
{name:'Farhan Ali'}))
RETURN [n IN nodes(path) | n.name] AS route, length(path) AS hops

// 5. Recommendation: co-authors of co-authors who have not yet worked with Asha
MATCH (me:Author {name:'Asha Verma'})-[:CO_AUTHOR]-(f)-[:CO_AUTHOR]-(rec)
WHERE rec <> me AND NOT (me)-[:CO_AUTHOR]-(rec)
RETURN rec.name AS recommended, count(f) AS common_collaborators
ORDER BY common_collaborators DESC
```

Output:

```
Created 13 nodes; created 14 relationships (10 WROTE, 4 CITES), 6 WORKS_AT and 5
CO_AUTHOR
Collaborators of Rahul Mehta : Asha Verma, Sneha Iyer
Papers citing 'Graph Indexing': Query Optimization
Authors per institution      : DAVV Indore 3, IIT Indore 3
Shortest path (Asha -> Farhan): [Asha Verma, Rahul Mehta, Sneha Iyer, Karan Joshi,
Meera Nair, Farhan Ali], hops = 5
Recommendation for Asha Verma : Sneha Iyer (1 common collaborator)
```

Result:

The research collaboration graph was created and the Cypher, shortest path and recommendation queries gave the expected results.

Conclusion:

Graph databases represent connected data naturally, and Cypher makes path and recommendation queries simple compared with SQL joins.

Experiment No. 13

Spatial Database using PostGIS

Aim:

To create a spatial database and to perform creation of spatial tables, point and polygon geometry, distance calculation, buffer analysis, spatial intersection and containment queries using PostGIS.

Software / Hardware Required:

PostgreSQL 15 with PostGIS 3.x extension, pgAdmin 4, QGIS (optional, for map display); PC with minimum 4 GB RAM.

Theory:

PostGIS adds spatial data types and functions to PostgreSQL. The geometry type stores points, lines and polygons in a coordinate system identified by an SRID (4326 is WGS 84 latitude-longitude). Important functions are ST_MakePoint and ST_GeomFromText (create geometry), ST_Distance (distance, in metres when the geography type is used), ST_Buffer (area within a given distance), ST_Intersects and ST_Intersection (overlap of two shapes) and ST_Contains / ST_Within (containment). A GiST index speeds up spatial queries.

Procedure / Algorithm:

1. Create the database and enable the PostGIS extension.
2. Create the tables library_branch (Point) and service_zone (Polygon) and insert the data for Indore.
3. Calculate the distance between branches using ST_Distance on geography.
4. Find the 1 km buffer of a branch, the intersection of two zones and the containment of branches in zones.
5. Create a GiST index and view the results in QGIS or pgAdmin's geometry viewer.

Code / Program:

```
CREATE DATABASE library_gis;
\c library_gis
CREATE EXTENSION postgis;

-- Spatial tables
CREATE TABLE library_branch (
  branch_id SERIAL PRIMARY KEY,
  name      VARCHAR(50),
  geom      GEOMETRY(Point, 4326)
);
CREATE TABLE service_zone (
  zone_id   SERIAL PRIMARY KEY,
  zone_name VARCHAR(40),
  area      GEOMETRY(Polygon, 4326)
);
CREATE INDEX idx_branch_geom ON library_branch USING GIST (geom);
CREATE INDEX idx_zone_area   ON service_zone   USING GIST (area);

-- Point geometry (longitude, latitude)
INSERT INTO library_branch (name, geom) VALUES
 ('Central Library',  ST_SetSRID(ST_MakePoint(75.8577, 22.7196), 4326)),
 ('Rajwada Branch',   ST_SetSRID(ST_MakePoint(75.8553, 22.7183), 4326)),
 ('Vijay Nagar Branch', ST_SetSRID(ST_MakePoint(75.8937, 22.7533), 4326)),
 ('Palasia Branch',   ST_SetSRID(ST_MakePoint(75.8830, 22.7244), 4326)),
 ('Bhawarkuan Branch', ST_SetSRID(ST_MakePoint(75.8700, 22.6970), 4326));

-- Polygon geometry
INSERT INTO service_zone (zone_name, area) VALUES
 ('Central Zone', ST_GeomFromText('POLYGON((75.84 22.70, 75.88 22.70, 75.88 22.74,
75.84 22.74, 75.84 22.70))', 4326)),
 ('East Zone',    ST_GeomFromText('POLYGON((75.87 22.71, 75.91 22.71, 75.91 22.77,
75.87 22.77, 75.87 22.71))', 4326));

-- Distance calculation (metres)
```

```
SELECT a.name AS from_branch, b.name AS to_branch,
       ROUND(ST_Distance(a.geom::geography, b.geom::geography)) AS distance_m
FROM library_branch a JOIN library_branch b ON a.branch_id < b.branch_id
ORDER BY distance_m;

-- Buffer analysis: branches within 3 km of the Central Library
SELECT b.name
FROM library_branch c, library_branch b
WHERE c.name = 'Central Library' AND b.name <> c.name
  AND ST_Intersects(b.geom, ST_Buffer(c.geom::geography, 3000)::geometry);
SELECT ST_AsText(ST_Buffer(geom::geography, 1000)::geometry) FROM library_branch WHERE
branch_id = 1;

-- Spatial intersection of two zones
SELECT ST_AsText(ST_Intersection(z1.area, z2.area)) AS overlap,
       ROUND((ST_Area(ST_Intersection(z1.area, z2.area)::geography) /
1000000)::numeric, 2) AS overlap_sq_km
FROM service_zone z1, service_zone z2
WHERE z1.zone_name = 'Central Zone' AND z2.zone_name = 'East Zone';

-- Containment queries
SELECT z.zone_name, b.name
FROM service_zone z JOIN library_branch b ON ST_Contains(z.area, b.geom)
ORDER BY z.zone_name, b.name;
SELECT name FROM library_branch
WHERE NOT EXISTS (SELECT 1 FROM service_zone z WHERE ST_Within(library_branch.geom,
z.area));
```

Output:

```
Distance (m)     : Central-Rajwada 285 | Central-Palasia 2649 | Central-Bhawarkuan
2812 | Central-Vijay Nagar 5260 | ... (approximate geodesic values)
Within 3 km of Central Library : Rajwada Branch, Palasia Branch, Bhawarkuan Branch
Intersection     : POLYGON((75.87 22.71, 75.88 22.71, 75.88 22.74, 75.87 22.74, 75.87
22.71)), about 3.4 sq km
Containment      : Central Zone -> Central Library, Rajwada Branch | East Zone ->
Palasia Branch, Vijay Nagar Branch
Outside all zones: Bhawarkuan Branch
```

Result:

The spatial database was created and the distance, buffer, intersection and containment queries were executed.

Conclusion:

PostGIS extends PostgreSQL with spatial types and functions, so location-based analysis of library branches can be done with ordinary SQL.